Introduction to Machine Learning

Lecture 21: Reinforcement Learning

Ali Bereyhi

ali.bereyhi@utoronto.ca

Department of Electrical and Computer Engineering

University of Toronto

Winter 2025

Today's Agenda: *Reinforcement Learning*

We finish our final journey on

Advanced Topics in Machine Learning

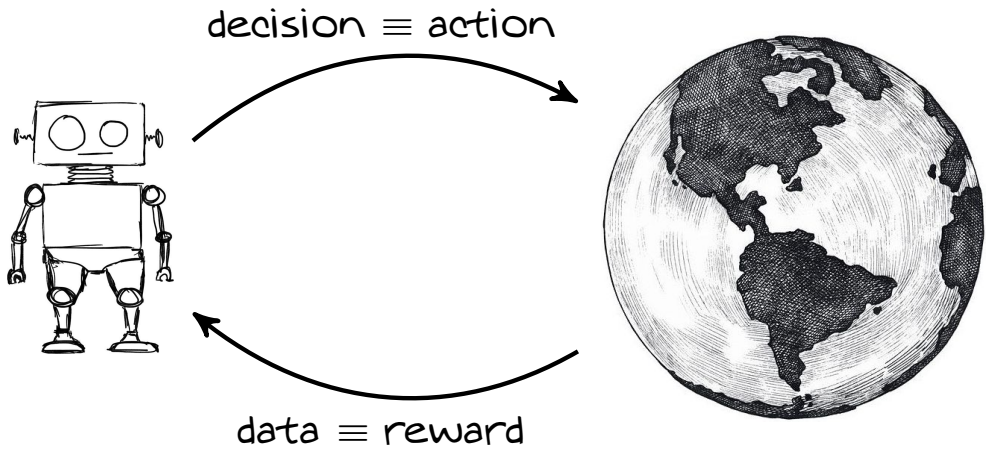
by getting briefly into

- *Reinforcement Learning*

What is Reinforcement Learning?

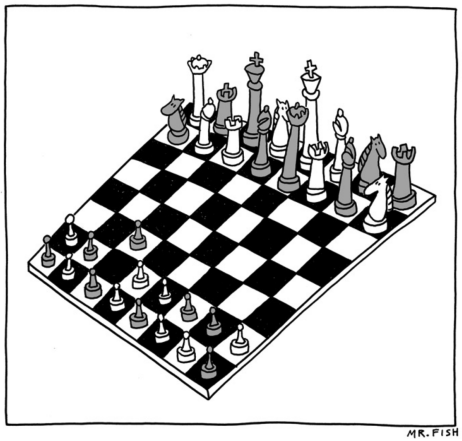
Reinforcement Learning

is about learning how to interact with environment



We learn many things in our life by this approach

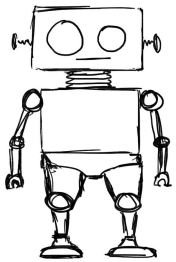
Learning to Play a Game



- *RL is not supervised*
- *RL does not have conventional dataset*
- *In RL time really matters*

Problem Components: *Environment*

A basic RL problem consists of an agent and environment

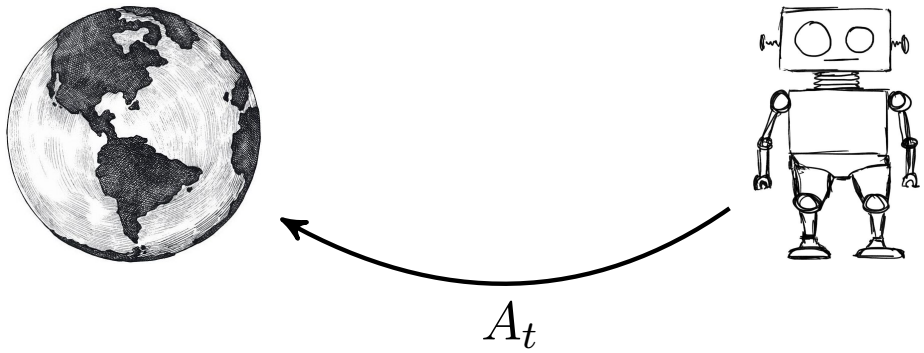


The environment is described by its state S_t

- *State can be the status of the game board*

Problem Components: *Agent*

Agent interacts with the environment

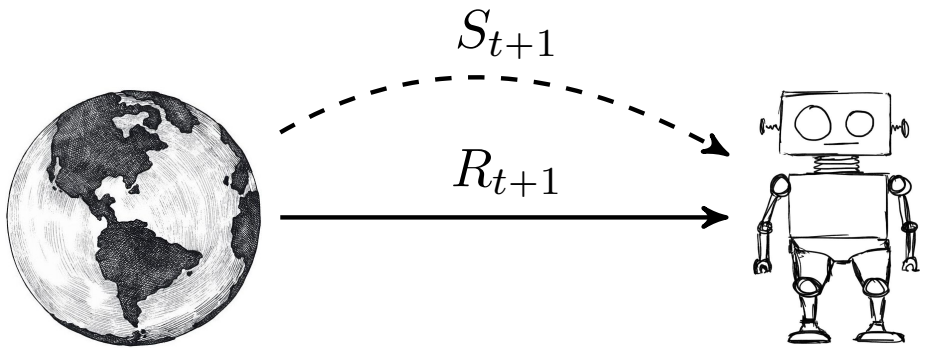


The agent performs action A_t at time t

- *State can be the status of the game board*

Interaction Model: *Transition and Reward*

As agent acts the environment changes its state and returns a reward

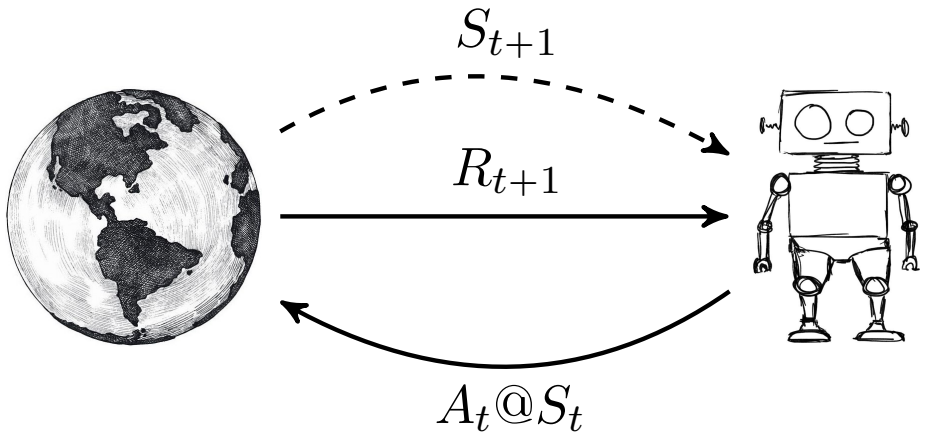


This transition can be modeled by a Markov process

$$P\left(S_{t+1}, R_{t+1} | S_t, A_t\right)$$

Objective: *Future Return*

At each time, the agent only cares about future

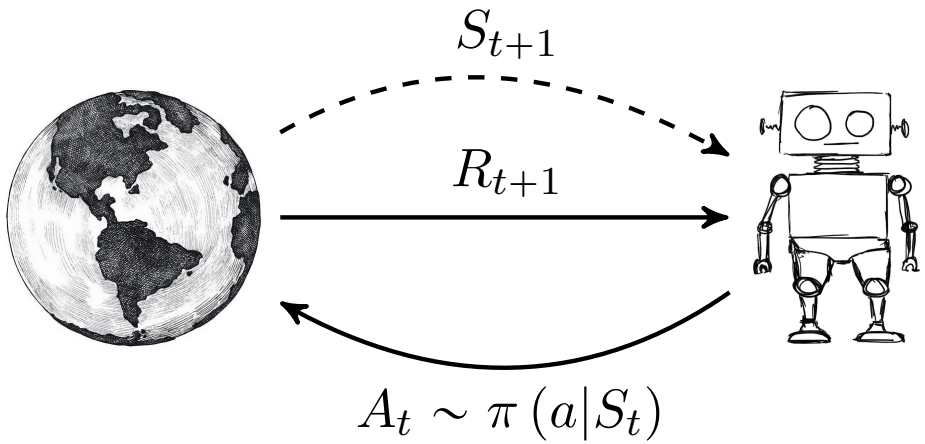


Through time, agents sees $A_0, S_0 \xrightarrow{R_1} A_1, S_1 \xrightarrow{R_2} A_2, S_2 \xrightarrow{R_3} \ldots$ *and wants to maximize its future return*

$$G_t = \sum_{i=0}^{\infty} R_{t+i+1}$$

Problem Components: *Policy*

So, the agent chooses a strategy or *policy* π



At each time it observes the state and samples an action from π

- *This policy can be deterministic or random*

Problem Components: *Optimal Policy*

The agent thinks statistically:

at each state, it wants to optimize its expected future return

Value Function

Say we start with state s and play with π, i.e.,

$$A_t, S_t = s \xrightarrow{R_{t+1}} A_{t+1}, S_{t+1} \xrightarrow{R_{t+2}} \dots$$

Then the value of this state with policy π is

$$v_\pi(s) = \mathbb{E}\{G_t | \pi, S_t = s\} = \mathbb{E}\left\{\sum_{i=1}^{\infty} R_{t+1} | \pi, S_t = s\right\}$$

Value tells us how much we gain in average starting at s and playing with π

Problem Components: *Optimal Policy*

The agent thinks statistically:

at each state, it wants to optimize its expected future return

This means that it looks for optimal policy $\pi^\star$

$$\pi^\star\left(\cdot|s\right) = \underset{\pi}{\operatorname{argmax}}\, v_\pi\left(s\right)$$

Problem Components: *Action-Value*

We can formulate the optimal policy using action-values

Action-Value Function

The action-value at pair (a, s) when we play with policy π is

$$q_\pi(a, s) = \mathbb{E}\{G_t | \pi, S_t = s, A_t = a\}$$

Action-value specifies the expected return when

we start at s and act a and keep playing with policy π

This is not hard to show that

$$v_\pi(s) = \sum_a \pi(a|s)\, q_\pi(a, s)$$

Problem Components: *Optimal Policy*

Key Observation

If we could find the optimal action-value; then, the problem is over!

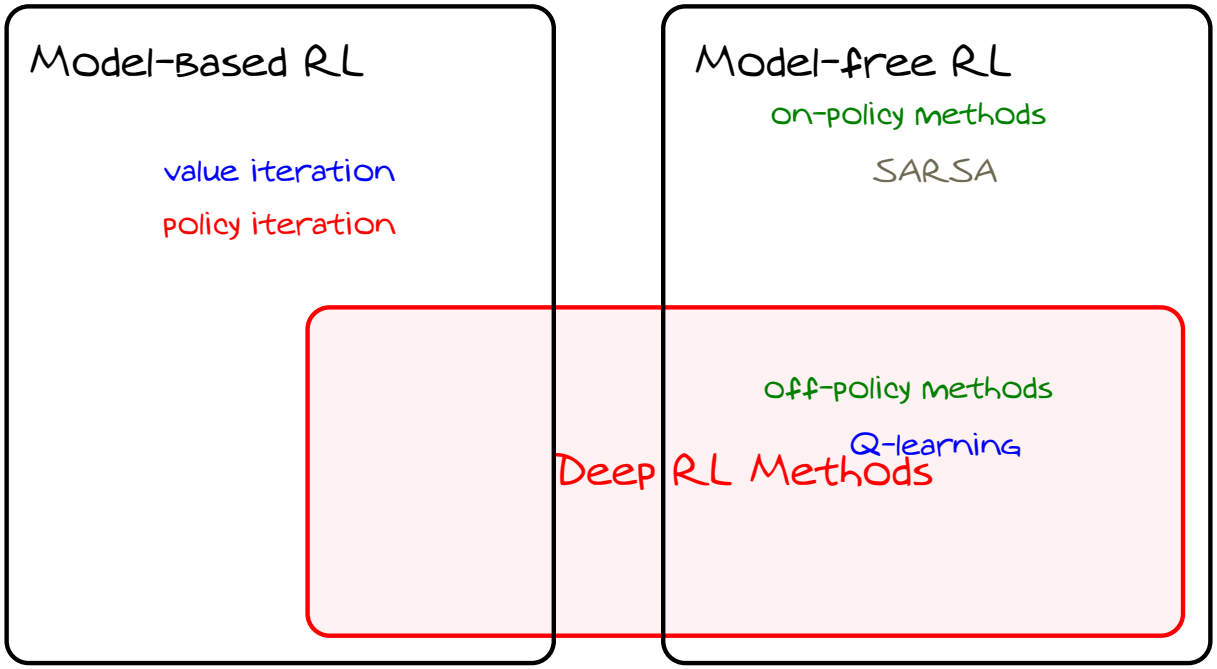
Say we know optimal action value at each point, i.e., $q_{\pi^\star}(a, s)$*: then,*

$$\pi^\star(a|s) = \begin{cases} 1 & a = \text{argmax}_x \, q_{\pi^\star}(x, s) \\ 0 & a \neq \text{argmax}_x \, q_{\pi^\star}(x, s) \end{cases}$$

Moral of Story

We may need to only look for optimal action-values

Overview of RL Algorithms



Known Environment

An ideal case is when

- *we observe the state completely*
- *we know that transition* $P\left(S_{t+1}, R_{t+1} | S_t, A_t\right)$ *of the Markov Chain*

In this case, we can compute all expectations!

Bellman Equation

Value of different states and actions are recursively related

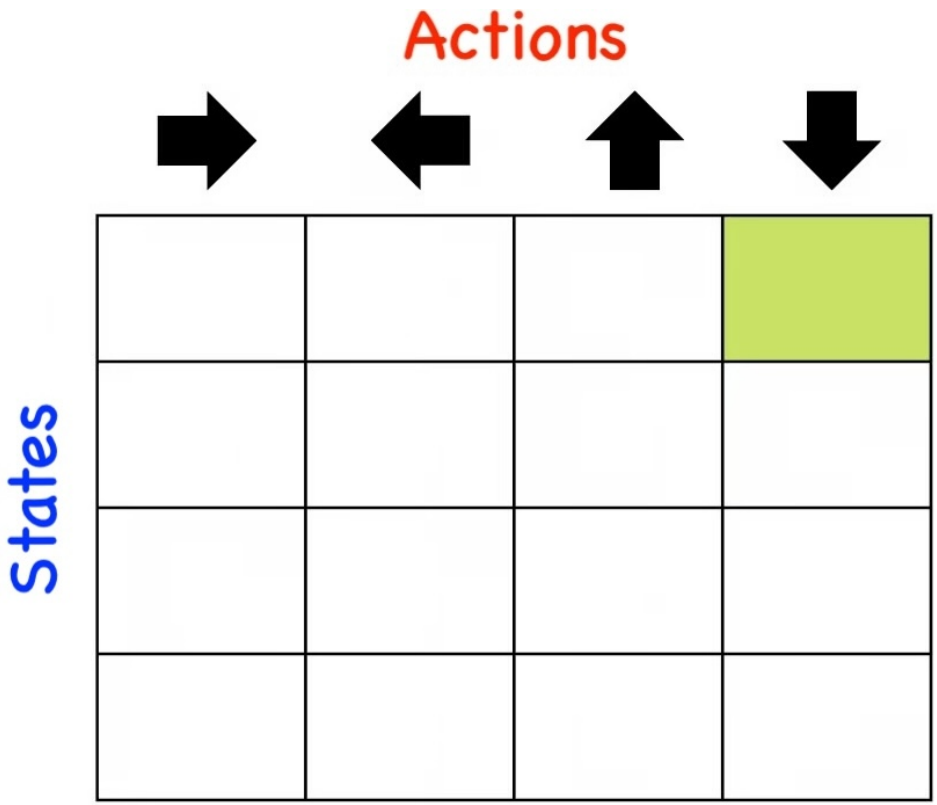
$$q_\pi\left(s, a\right) = \bar{\mathcal{R}}\left(s, a\right) + \sum_{s'} v_\pi\left(s'\right) P\left(s' | s, a\right)$$

where $\bar{\mathcal{R}}\left(s, a\right)$ *is the expected immediate reward at state* s *and action* a

Finding Optimal Policy

We can use this recursive equation to find optimal action-value

- *Start with initial policy and iterate by recursion till converge*
- *We find the optimal policy out of the optimal action-value*



Value and Policy Iteration

There are two well-known iterative approaches

- *Policy Iteration*
  - ↳ *It updates values and policy iteratively*
  - ↳ *It is computationally more complex*
  - ↳ *It can give better policies once converged*
- *Value Iteration*
  - ↳ *It works with values and find the policy at the end*
  - ↳ *It is computationally less complex*
  - ↳ *It might end with less accurate policy once converged*

We can also make a trade-off by using *generalized policy iteration*

Unknown Environment

In most practical cases

- *we observe the state partially*
- *we don't really know the transition* $P\left(S_{t+1}, R_{t+1} | S_t, A_t\right)$

In this case, we cannot compute the expectations, e.g.,

$$q_\pi\left(s, a\right) = \mathbb{E}\left\{G_t | \pi S_t = s, A_t = a\right\}$$
$$q_\pi\left(s, a\right) = \bar{\mathcal{R}}\left(s, a\right) + \sum_{s'} v_\pi\left(s'\right) P\left(s' | s, a\right)$$

The idea in this case is to estimate these expectation, using the fact that

$$\mathbb{E}\left\{X\right\} \approx \frac{1}{N} \sum_{n=1}^{N} X_n$$

Estimating Value Function

There are two main approaches:

1. *We can directly start from the definition, i.e.,*

$$q_\pi(s, a) = \mathbb{E}\{G_t | \pi S_t = s, A_t = a\}$$

and estimate by sampling $\rightsquigarrow$ *Monte-Carlo method*

2. *We can try to estimate expectations in Bellman equations, i.e.,*

$$q_\pi(s, a) = \bar{\mathcal{R}}(s, a) + \sum_{s'} v_\pi(s') P(s' | s, a)$$

and use recursion $\rightsquigarrow$ *Temporal-Difference method*

We typically use the second one as it is more efficient sample-wise

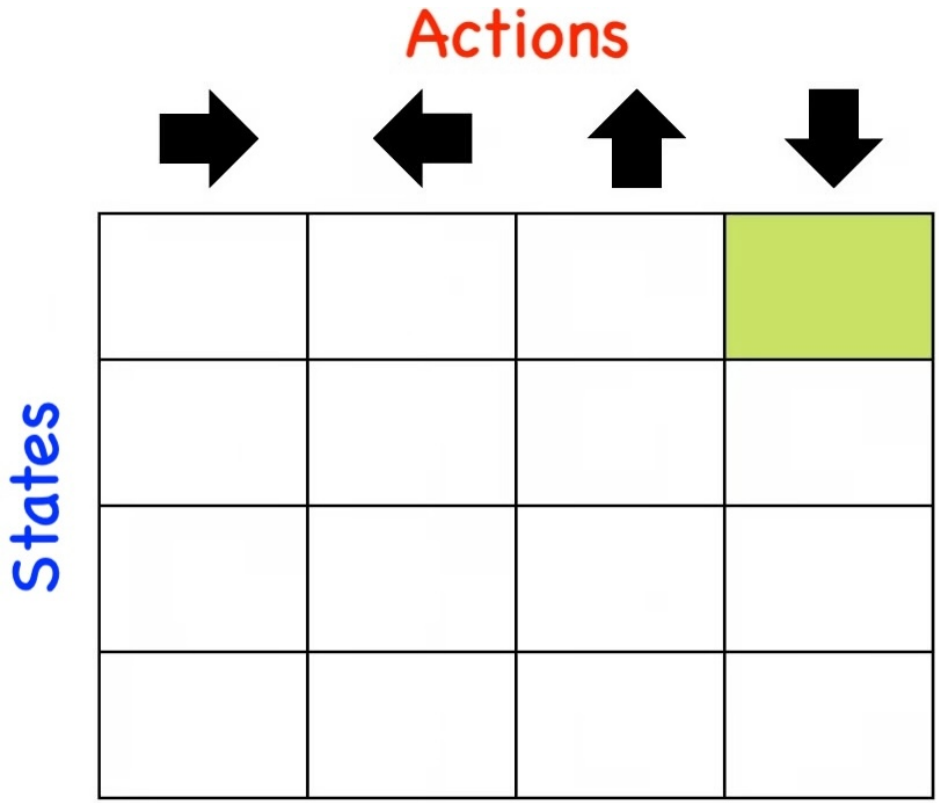
Overview of Model Free Algorithms

We can also take two approaches for sampling:

- *We sample with the same policy π that we compute its action-values*
  - ↳ *On-policy methods*
  - ↳ *Most famous one ⇝ SARSA*
- *We can sample with policy π and compute action-values of another policy*
  - ↳ *This can be done using the idea of importance sampling*
  - ↳ *Off-policy methods*
  - ↳ *Most famous one ⇝ Q-learning*

Dimensionality Issue

Classical RL approaches are called tabular: *they estimate values in the table*



But in practice this table can be huge and we cannot collect enough samples

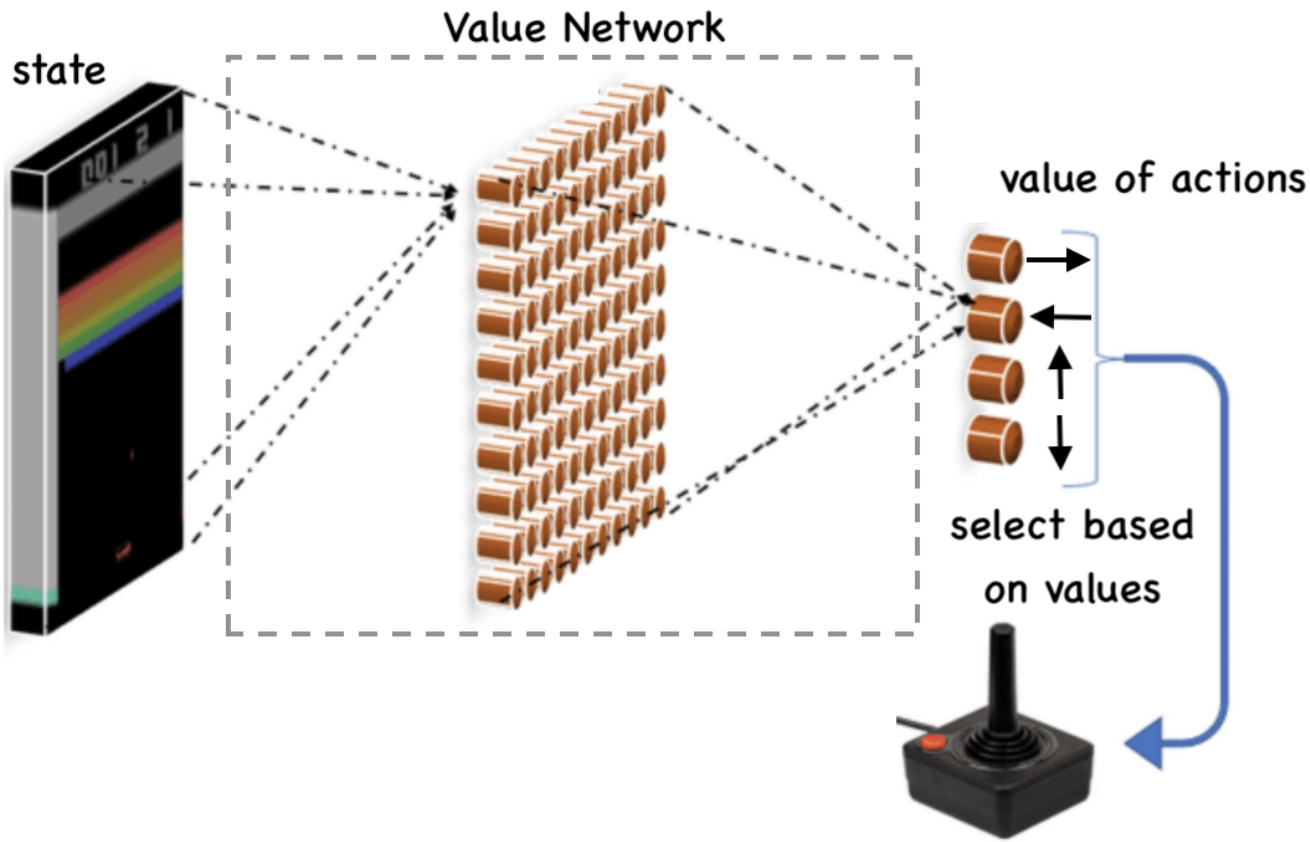
Using Deep Models to Estimate

Deep RL suggests to *use deep NN to approximate these tabular functions*

- *we can take a samples trajectories*
  - *For a few of them, we can estimate the value*
- *we then treat them as labeled samples*
  - *We have sample states and actions as input*
  - *We have their values as the label*
- *we train a deep NN to learn the value function*

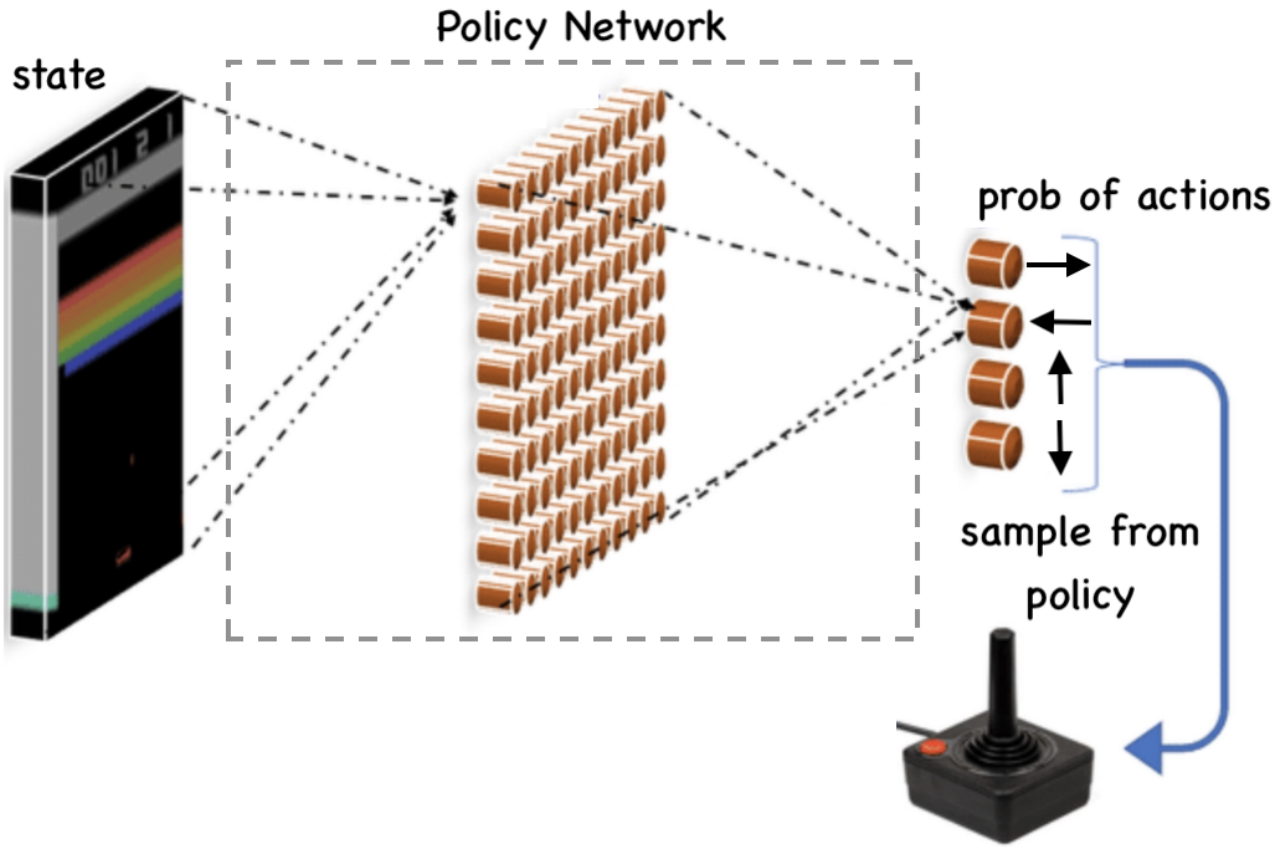
Deep Value Networks

The most basic approach is to learn the action-value function



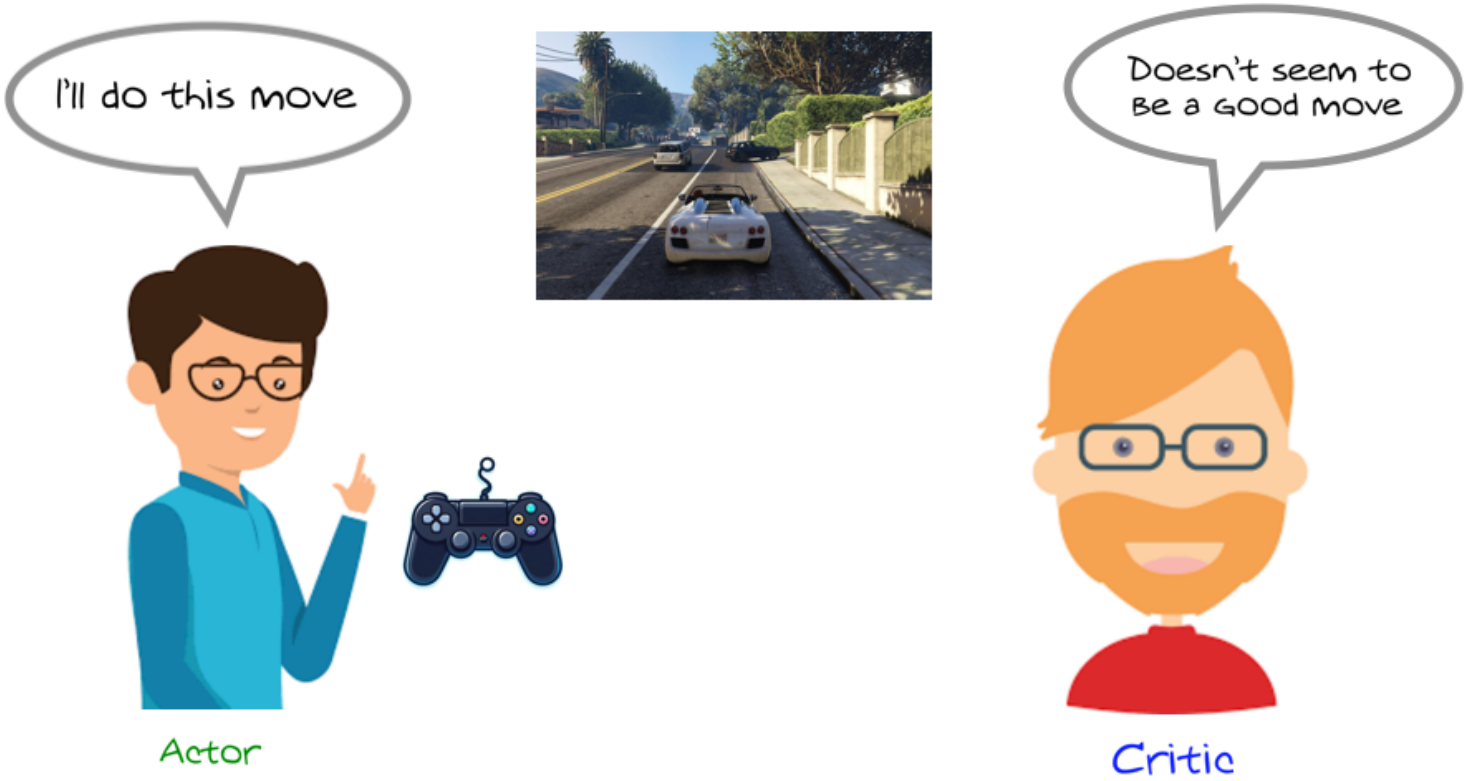
Deep Policy Networks

We could though try to learn directly the policy



Actor-Critic Approaches

We can combine both policy and value networks



Further Read

- Textbook by Richard Sutton
- Spinning up Deep RL by Open AI